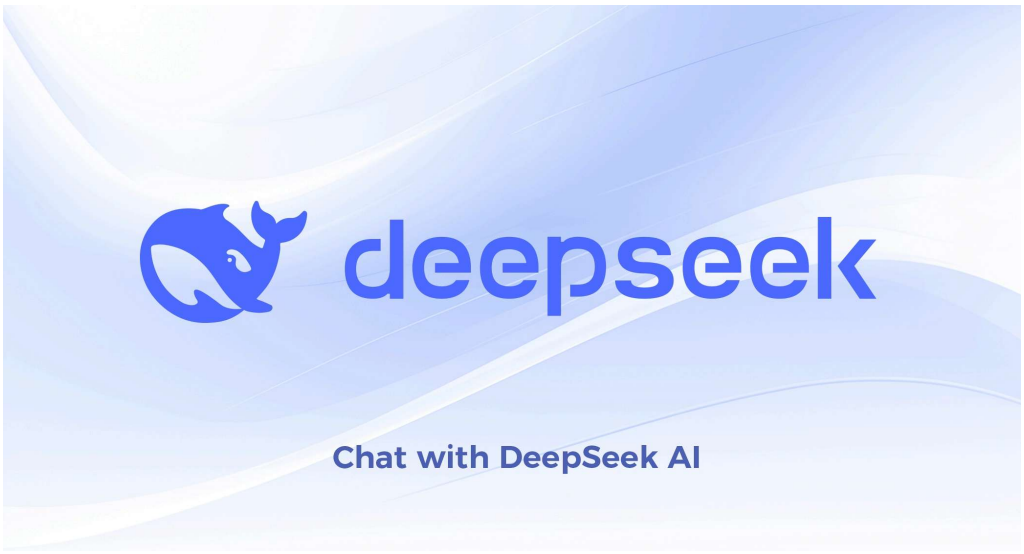
Real-Life Exception Handling Examples for Interviews



I'll provide a comprehensive guide to exception handling concepts with code examples and explanations.

1. What are Exceptions?

Exceptions are unexpected events that occur during program execution that disrupt the normal flow of instructions.

python

```
number = int("abc")
print("This line never executes")
```

2. Basic Try-Except Block

Python Example

python

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except:
    print("Something went wrong!")
```

Java Example

java

```
try {
    int number = Integer.parseInt("abc");
    int result = 10 / number;
    System.out.println("Result: " + result);
} catch (Exception e) {
    System.out.println("Something went wrong!");
}
```

3. Catching Specific Exceptions

Python

python

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")
except ValueError:
    print("Please enter a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Java

java

```
try {
    int[] numbers = {1, 2, 3};
    System.out.println(numbers[5]);
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Array index out of bounds: " + e.getMessage());
} catch (ArithmeticException e) {
    System.out.println("Arithmetic error: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Generic error: " + e.getMessage());
}
```

4. Try-Except-Else-Finally

Python

python

```
def read_file(filename):
    file = None
    try:
        file = open(filename, 'r')
        content = file.read()
    except FileNotFoundError:
        print(f"File '{filename}' not found")
    except PermissionError:
        print(f"Permission denied to read '{filename}'")
    else:

        print(f"File read successfully. Content length: {len(content)}")
        return content
    finally:

        if file:
            file.close()
            print("File closed")
```

Java

java

```
import java.io.*;

public void readFile(String filename) {
    BufferedReader reader = null;
    try {
        reader = new BufferedReader(new FileReader(filename));
        String line;
        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }
    } catch (FileNotFoundException e) {
        System.out.println("File not found: " + filename);
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    } finally {
        try {
            if (reader != null) {
                reader.close();
                System.out.println("Reader closed");
            }
        } catch (IOException e) {
            System.out.println("Error closing reader");
        }
    }
}
```

5. Custom Exceptions

Python

python

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"Insufficient funds: Balance ${balance}, "
                         f"Attempted withdrawal ${amount}")

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount
        return amount


account = BankAccount(100)
try:
    account.withdraw(150)
except InsufficientFundsError as e:
    print(f"Error: {e}")
    print(f"Missing: ${e.amount - e.balance}")
```

Java

java

```
class InsufficientFundsException extends Exception {
    private double balance;
    private double amount;

    public InsufficientFundsException(double balance, double amount) {
        super("Insufficient funds: Balance $" + balance +
              ", Attempted withdrawal $" + amount);
        this.balance = balance;
        this.amount = amount;
    }

    public double getMissing() {
        return amount - balance;
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(balance, amount);
        }
        balance -= amount;
    }
}


BankAccount account = new BankAccount(100);
try {
    account.withdraw(150);
} catch (InsufficientFundsException e) {
    System.out.println(e.getMessage());
    System.out.println("Missing: $" + e.getMissing());
}
```

6. Exception Chaining

Python

python

```
def fetch_user_data(user_id):
    try:
        response = api_call(user_id)
        return response
    except ConnectionError as e:
        raise RuntimeError("Failed to fetch user data") from e

def process_user(user_id):
    try:
        data = fetch_user_data(user_id)
        process(data)
    except RuntimeError as e:
        print(f"Processing failed: {e}")
        print(f"Original cause: {e.__cause__}")
```

Java

java

```
public void fetchUserData(int userId) throws DataAccessException {
    try {
        Response response = apiCall(userId);
        return response;
    } catch (ConnectionException e) {
        throw new DataAccessException("Failed to fetch user data", e);
    }
}


try {
    fetchUserData(123);
} catch (DataAccessException e) {
    System.out.println("Processing failed: " + e.getMessage());
    System.out.println("Original cause: " + e.getCause());
}
```

7. Context Managers (Python) / Try-with-Resources (Java)

Python

python

```
class DatabaseConnection:
    def __init__(self, db_name):
        self.db_name = db_name

    def __enter__(self):
        print(f"Connecting to {self.db_name}")
        self.connection = self._connect()
        return self.connection

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Closing connection to {self.db_name}")
        self.connection.close()
        return False

    def _connect(self):
        return "dummy_connection"

    def execute_query(self, query):
        if "invalid" in query.lower():
            raise ValueError("Invalid query")
        return f"Results for: {query}"


try:
    with DatabaseConnection("mydb") as db:
        results = db.execute_query("SELECT * FROM users")
        print(results)
except ValueError as e:
    print(f"Query error: {e}")
```

Java

java

```
public void processFile(String filename) {
    try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
        String line;
        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }
    } catch (FileNotFoundException e) {
        System.out.println("File not found: " + filename);
    } catch (IOException e) {
        System.out.println("Error reading file: " + e.getMessage());
    }

}
```

8. Exception Handling Best Practices

Python

python

```
import logging


logging.basicConfig(level=logging.ERROR)

def process_data(data):




    connection = None
    try:
        connection = create_connection()
        validate_data(data)
        result = transform_data(data)
        save_result(connection, result)
        return result
    except ValidationError as e:
        logging.error(f"Data validation failed: {e}")
        raise
    except ConnectionError as e:
        logging.error(f"Connection failed: {e}")
        raise RuntimeError("Database connection error") from e
    except Exception as e:
        logging.exception("Unexpected error occurred")
        raise
    finally:
        if connection:
            connection.close()


import time
from functools import wraps

def retry(max_attempts=3, delay=1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts - 1:
                        raise
                    print(f"Attempt {attempt + 1} failed: {e}")
                    time.sleep(delay)
            return None
        return wrapper
    return decorator
```

```
@retry(max_attempts=3, delay=2)
def unstable_network_call():
    import random
    if random.random() < 0.7:
        raise ConnectionError("Network timeout")
    return "Success"


try:
    result = unstable_network_call()
    print(f"Result: {result}")
except ConnectionError:
    print("Failed after all retries")
```

Java Best Practices

java

```
import java.util.logging.*;

public class ExceptionBestPractices {
    private static final Logger logger = Logger.getLogger(ExceptionBestPractices.class.getName());




    public void processUserInput(String input) throws ValidationException {
        if (input == null) {
            throw new ValidationException("Input cannot be null");
        }
        if (input.isEmpty()) {
            throw new ValidationException("Input cannot be empty");
        }

    }


    public void saveToDatabase(User user) throws ServiceException {
        try {
            databaseConnection.save(user);
        } catch (SQLException e) {
            logger.log(Level.SEVERE, "Database error while saving user", e);
            throw new ServiceException("Failed to save user", e);
        }
    }


    public void readData() {
        Connection conn = null;
        try {
            conn = getConnection();

        } catch (SQLException e) {
            logger.log(Level.SEVERE, "Error accessing database", e);
        } finally {
            if (conn != null) {
                try {
                    conn.close();
                } catch (SQLException e) {
                    logger.log(Level.WARNING, "Error closing connection", e);
                }
            }
        }
    }
}
```

9. Common Exception Types

Python Exceptions

python

```
"""
BaseException
 ├── SystemExit
 ├── KeyboardInterrupt
 ├── GeneratorExit
 └── Exception
     ├── ArithmeticError
     │    ├── ZeroDivisionError
     │    └── OverflowError
     ├── AssertionError
     ├── AttributeError
     ├── ImportError
     ├── IndexError
     ├── KeyError
     ├── NameError
     ├── OSError
     │    ├── FileNotFoundError
     │    └── PermissionError
     ├── TypeError
     ├── ValueError
     └── RuntimeError
"""


def demonstrate_exceptions():
    try:

        x = 1 / 0
    except ZeroDivisionError:
        print("Cannot divide by zero")

    try:

        list[999]
    except IndexError:
        print("List index out of range")

    try:

        dict['nonexistent']
    except KeyError:
        print("Key not found in dictionary")

    try:

        'string' + 123
    except TypeError:
        print("Type mismatch")
```

```
    try:

        int('abc')
    except ValueError:
        print("Invalid literal for conversion")
```

Java Exceptions

java

```
public class ExceptionTypes {
    public void demonstrateExceptions() {

        try {
            FileReader file = new FileReader("nonexistent.txt");
        } catch (FileNotFoundException e) {
            System.out.println("Checked exception: File not found");
        }


        try {
            int[] arr = new int[5];
            arr[10] = 50;
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Unchecked exception: " + e.getMessage());
        }


        try {
            String str = null;
            str.length();
        } catch (NullPointerException e) {
            System.out.println("Null pointer exception");
        }


        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic exception: " + e.getMessage());
        }
    }
}
```

10. Advanced Patterns

Python - Context Manager with Error Handling

python

```
from contextlib import contextmanager

@contextmanager
def transaction(connection):

    print("Starting transaction")
    try:
        yield connection

        print("Committing transaction")
        connection.commit()
    except Exception:

        print("Rolling back transaction")
        connection.rollback()
        raise
    finally:

        print("Cleaning up resources")


class Connection:
    def commit(self):
        print("DB commit")
    def rollback(self):
        print("DB rollback")
    def execute(self, query):
        print(f"Executing: {query}")
        if "error" in query:
            raise ValueError("Query failed")

conn = Connection()
try:
    with transaction(conn) as t:
        t.execute("INSERT INTO users VALUES (1, 'John')")
        t.execute("error")
except ValueError as e:
    print(f"Transaction failed: {e}")
```

Summary of Key Concepts:

1. **Exception Hierarchy**: Exceptions form a tree structure

2. **Specific vs. Generic**: Catch specific exceptions before generic ones
3. **Resource Management**: Use finally or context managers
4. **Custom Exceptions**: Create meaningful exception types
5. **Exception Chaining**: Preserve the original error context
6. **Don't Swallow Exceptions**: Always log or handle appropriately
7. **Fail Fast**: Validate inputs early and raise exceptions immediately

Exception handling makes programs robust and maintainable by separating error-handling logic from normal program flow.

real life example and code for interview

I'll provide production-ready examples that demonstrate real-world scenarios interviewers look for.

1. E-Commerce Order Processing System

This is a classic interview example showing multiple exception handling patterns.

python

```
import logging
from datetime import datetime
from typing import List, Optional
from decimal import Decimal

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


class OrderProcessingError(Exception):
    """Base exception for order processing"""
    pass

class InsufficientStockError(OrderProcessingError):
    def __init__(self, product_id: str, requested: int, available: int):
        self.product_id = product_id
        self.requested = requested
        self.available = available
        super().__init__(
            f"Product {product_id}: Requested {requested}, Available {available}"
        )

class PaymentFailedError(OrderProcessingError):
    def __init__(self, order_id: str, reason: str):
        self.order_id = order_id
        self.reason = reason
        super().__init__(f"Payment failed for order {order_id}: {reason}")

class ShippingNotAvailableError(OrderProcessingError):
    def __init__(self, pincode: str):
        self.pincode = pincode
        super().__init__(f"Shipping not available for pincode: {pincode}")


class Product:
    def __init__(self, id: str, name: str, price: Decimal, stock: int):
        self.id = id
        self.name = name
        self.price = price
        self.stock = stock

class Order:
    def __init__(self, id: str, user_id: str, items: List[dict]):
        self.id = id
        self.user_id = user_id
        self.items = items
        self.status = "PENDING"
        self.total = Decimal('0')
        self.created_at = datetime.now()
```

```
class OrderService:
    def __init__(self):
        self.products_db = {
            "P001": Product("P001", "iPhone 15", Decimal('999.99'), 10),
            "P002": Product("P002", "MacBook Pro", Decimal('1999.99'), 5),
            "P003": Product("P003", "AirPods", Decimal('199.99'), 0)
        }
        self.orders_db = {}

    def process_order(self, order: Order) -> dict:
        """Main order processing method with comprehensive error handling"""
        logger.info(f"Processing order {order.id} for user {order.user_id}")

        try:

            self._validate_and_reserve_stock(order)


            order.total = self._calculate_total(order)


            payment_result = self._process_payment(order)


            self._validate_shipping(order)


            tracking_id = self._create_shipment(order)


            order.status = "CONFIRMED"
            self.orders_db[order.id] = order

            logger.info(f"Order {order.id} processed successfully")
            return {
                "success": True,
                "order_id": order.id,
                "tracking_id": tracking_id,
                "total": str(order.total),
                "status": order.status
            }

        except InsufficientStockError as e:
            logger.error(f"Stock error for order {order.id}: {e}")
            order.status = "FAILED"
            return {
                "success": False,
                "error": "INSUFFICIENT_STOCK",
                "message": str(e),
                "product_id": e.product_id,
```

```
                "suggestion": f"Only {e.available} items available"
            }

        except PaymentFailedError as e:
            logger.error(f"Payment error for order {order.id}: {e}")
            self._release_stock(order)
            order.status = "PAYMENT_FAILED"
            return {
                "success": False,
                "error": "PAYMENT_FAILED",
                "message": str(e),
                "suggestion": "Please try another payment method"
            }

        except ShippingNotAvailableError as e:
            logger.error(f"Shipping error for order {order.id}: {e}")
            self._refund_payment(order)
            self._release_stock(order)
            order.status = "FAILED"
            return {
                "success": False,
                "error": "SHIPPING_UNAVAILABLE",
                "message": str(e),
                "suggestion": "Try different delivery address"
            }

        except Exception as e:
            logger.exception(f"Unexpected error processing order {order.id}")

            self._perform_full_rollback(order)
            return {
                "success": False,
                "error": "INTERNAL_ERROR",
                "message": "An unexpected error occurred. Our team has been notified."
            }

    def _validate_and_reserve_stock(self, order: Order):
        """Validate and reserve stock with atomic operation simulation"""
        for item in order.items:
            product_id = item['product_id']
            quantity = item['quantity']

            product = self.products_db.get(product_id)
            if not product:
                raise InsufficientStockError(product_id, quantity, 0)

            if product.stock < quantity:
                raise InsufficientStockError(product_id, quantity, product.stock)


            product.stock -= quantity
```

```
        logger.info(f"Reserved {quantity} units of {product_id}")

    def _calculate_total(self, order: Order) -> Decimal:
        """Calculate order total with tax"""
        subtotal = Decimal('0')
        for item in order.items:
            product = self.products_db[item['product_id']]
            subtotal += product.price * item['quantity']


        tax = subtotal * Decimal('0.10')
        return subtotal + tax

    def _process_payment(self, order: Order) -> dict:
        """Simulate payment processing with retry logic"""
        max_retries = 3
        for attempt in range(max_retries):
            try:

                if order.total > Decimal('5000'):
                    raise PaymentFailedError(order.id, "Amount exceeds limit")

                logger.info(f"Payment processed: ${order.total}")
                return {"transaction_id": f"TXN_{order.id}", "status": "SUCCESS"}

            except PaymentFailedError:
                raise

            except Exception as e:
                if attempt == max_retries - 1:
                    raise PaymentFailedError(order.id, f"Failed after {max_retries} attempts")
                logger.warning(f"Payment attempt {attempt + 1} failed, retrying...")

        raise PaymentFailedError(order.id, "All payment attempts exhausted")

    def _validate_shipping(self, order: Order):
        """Validate shipping availability"""

        serviceable_pincodes = ["10001", "20002", "30003"]
        delivery_pincode = "99999"

        if delivery_pincode not in serviceable_pincodes:
            raise ShippingNotAvailableError(delivery_pincode)

    def _create_shipment(self, order: Order) -> str:
        """Create shipment and return tracking ID"""

        return f"TRACK_{order.id}_{datetime.now().timestamp()}"

    def _release_stock(self, order: Order):
        """Rollback stock reservation"""
```

```
        for item in order.items:
            product = self.products_db[item['product_id']]
            product.stock += item['quantity']
            logger.info(f"Released {item['quantity']} units of {item['product_id']}")

    def _refund_payment(self, order: Order):
        """Process refund"""
        logger.info(f"Refund initiated for order {order.id}")

    def _perform_full_rollback(self, order: Order):
        """Complete rollback of all operations"""
        try:
            self._refund_payment(order)
        except Exception as e:
            logger.error(f"Refund failed during rollback: {e}")

        try:
            self._release_stock(order)
        except Exception as e:
            logger.error(f"Stock release failed during rollback: {e}")


def demonstrate_order_processing():
    service = OrderService()


    print("\n=== Scenario 1: Successful Order ===")
    order1 = Order("ORD001", "USER123", [
        {"product_id": "P001", "quantity": 1},
        {"product_id": "P002", "quantity": 1}
    ])
    result1 = service.process_order(order1)
    print(f"Result: {result1}")


    print("\n=== Scenario 2: Insufficient Stock ===")
    order2 = Order("ORD002", "USER456", [
        {"product_id": "P003", "quantity": 5}
    ])
    result2 = service.process_order(order2)
    print(f"Result: {result2}")


    print("\n=== Scenario 3: Payment Failed ===")
    order3 = Order("ORD003", "USER789", [
        {"product_id": "P001", "quantity": 10}
    ])
    result3 = service.process_order(order3)
    print(f"Result: {result3}")
```

```
if __name__ == "__main__":
    demonstrate_order_processing()
```

2. File Processing System with Retry and Recovery

This example shows handling transient failures in data processing.

python

```
import json
import time
from pathlib import Path
from typing import Dict, Any
from functools import wraps

class DataProcessingError(Exception):
    pass

class FileCorruptedError(DataProcessingError):
    pass

class TransformationError(DataProcessingError):
    pass


def retry_with_backoff(max_retries=3, initial_delay=1, backoff_factor=2):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            delay = initial_delay
            last_exception = None

            for attempt in range(max_retries):
                try:
                    return func(*args, **kwargs)
                except (ConnectionError, TimeoutError) as e:
                    last_exception = e
                    if attempt < max_retries - 1:
                        print(f"Attempt {attempt + 1} failed: {e}. Retrying in {delay}s...")
                        time.sleep(delay)
                        delay *= backoff_factor
                    else:
                        raise DataProcessingError(f"Failed after {max_retries} attempts") from e
                except Exception as e:
                    raise

            raise last_exception
        return wrapper
    return decorator

class DataProcessor:
    def __init__(self, input_dir: str, output_dir: str, error_dir: str):
        self.input_dir = Path(input_dir)
        self.output_dir = Path(output_dir)
        self.error_dir = Path(error_dir)
        self.processed_files = set()
        self.failed_files = {}
```

```
        self.output_dir.mkdir(exist_ok=True)
        self.error_dir.mkdir(exist_ok=True)

    @retry_with_backoff(max_retries=3, initial_delay=2)
    def fetch_data_from_api(self, file_id: str) -> Dict[str, Any]:
        """Simulate fetching additional data from external API"""

        import random
        if random.random() < 0.3:
            raise ConnectionError("API temporarily unavailable")

        return {"metadata": f"enrichment_{file_id}", "timestamp": time.time()}

    def process_file(self, filepath: Path) -> Dict[str, Any]:
        """Process a single file with comprehensive error handling"""
        filename = filepath.name
        print(f"\nProcessing: {filename}")

        try:

            try:
                with open(filepath, 'r') as f:
                    data = json.load(f)
            except json.JSONDecodeError as e:
                raise FileCorruptedError(f"Invalid JSON format: {e}")
            except FileNotFoundError:
                raise FileCorruptedError(f"File not found: {filename}")


            self._validate_schema(data, filename)


            enriched_data = self.fetch_data_from_api(data.get('id', filename))


            transformed_data = self._transform_data(data, enriched_data)


            output_path = self.output_dir / f"processed_{filename}"
            with open(output_path, 'w') as f:
                json.dump(transformed_data, f, indent=2)


            self.processed_files.add(filename)
            print(f"✓ Successfully processed: {filename}")

            return {"status": "success", "file": filename, "output": str(output_path)}

        except FileCorruptedError as e:
            print(f"✗ Corrupted file: {e}")
            self._handle_failed_file(filepath, "CORRUPTED", str(e))
```

```
            return {"status": "error", "file": filename, "error": str(e)}

        except DataProcessingError as e:
            print(f"✗ Processing error: {e}")
            self._handle_failed_file(filepath, "PROCESSING_ERROR", str(e))
            return {"status": "error", "file": filename, "error": str(e)}

        except Exception as e:
            print(f"✗ Unexpected error: {e}")
            self._handle_failed_file(filepath, "UNKNOWN", str(e))
            return {"status": "error", "file": filename, "error": "Unexpected error occurred"}

    def _validate_schema(self, data: Dict, filename: str):
        """Validate data schema"""
        required_fields = ['id', 'type', 'payload']
        missing = [field for field in required_fields if field not in data]
        if missing:
            raise FileCorruptedError(f"Missing required fields in {filename}: {missing}")

    def _transform_data(self, data: Dict, enriched: Dict) -> Dict:
        """Transform and enrich data"""
        try:
            return {
                "id": data['id'],
                "type": data['type'],
                "payload": data['payload'],
                "metadata": enriched.get('metadata'),
                "processed_at": enriched.get('timestamp'),
                "source": "batch_processor_v1"
            }
        except Exception as e:
            raise TransformationError(f"Data transformation failed: {e}")

    def _handle_failed_file(self, filepath: Path, error_type: str, error_msg: str):
        """Move failed file to error directory and log"""
        error_path = self.error_dir / filepath.name
        filepath.rename(error_path)

        self.failed_files[filepath.name] = {
            "error_type": error_type,
            "error_message": error_msg,
            "moved_to": str(error_path)
        }


        error_log_path = self.error_dir / f"{filepath.name}.error.log"
        with open(error_log_path, 'w') as f:
            f.write(f"Error Type: {error_type}\n")
            f.write(f"Error Message: {error_msg}\n")
            f.write(f"Timestamp: {time.time()}\n")
```

```
    def generate_report(self) -> Dict:
        """Generate processing report"""
        return {
            "total_processed": len(self.processed_files),
            "total_failed": len(self.failed_files),
            "processed_files": list(self.processed_files),
            "failed_files": self.failed_files,
            "success_rate": f"{(len(self.processed_files) / (len(self.processed_files) + len(self.failed_files)) *
100):.2f}%"
        }


def run_file_processing_demo():

    test_dir = Path("test_data")
    test_dir.mkdir(exist_ok=True)


    valid_data = {
        "id": "FILE001",
        "type": "customer_order",
        "payload": {"customer": "John Doe", "amount": 150}
    }
    with open(test_dir / "valid_order.json", 'w') as f:
        json.dump(valid_data, f)


    with open(test_dir / "corrupted.json", 'w') as f:
        f.write("{invalid json content")


    incomplete_data = {"id": "FILE003"}
    with open(test_dir / "incomplete.json", 'w') as f:
        json.dump(incomplete_data, f)


    processor = DataProcessor(
        input_dir="test_data",
        output_dir="processed",
        error_dir="errors"
    )


    results = []
    for filepath in test_dir.glob("*.json"):
        result = processor.process_file(filepath)
        results.append(result)


    print("\n" + "="*50)
    print("PROCESSING REPORT")
```

```
    print("="*50)
    report = processor.generate_report()
    for key, value in report.items():
        print(f"{key}: {value}")

if __name__ == "__main__":
    run_file_processing_demo()
```

3. Database Transaction Manager

This shows proper transaction management with savepoints.

python

```
import sqlite3
from contextlib import contextmanager
from typing import List, Tuple, Any
from datetime import datetime

class DatabaseError(Exception):
    pass

class ConstraintViolationError(DatabaseError):
    pass

class ConnectionPoolExhaustedError(DatabaseError):
    pass

@contextmanager
def database_transaction(connection, isolation_level="DEFERRED"):
    """Context manager for database transactions with automatic rollback"""
    cursor = connection.cursor()


    cursor.execute(f"PRAGMA read_uncommitted = {'true' if isolation_level == 'READ_UNCOMMITTED' else
'false'}")


    savepoint_name = f"sp_{datetime.now().timestamp()}"
    cursor.execute(f"SAVEPOINT {savepoint_name}")

    try:
        yield cursor

        cursor.execute(f"RELEASE SAVEPOINT {savepoint_name}")
        connection.commit()
        print("✓ Transaction committed successfully")

    except Exception as e:

        cursor.execute(f"ROLLBACK TO SAVEPOINT {savepoint_name}")
        print(f"✗ Transaction rolled back: {e}")
        raise DatabaseError(f"Transaction failed: {e}") from e

    finally:
        cursor.close()

class UserRepository:
    def __init__(self, db_path: str):
        self.db_path = db_path
        self.connection = None
        self._initialize_database()

    def _initialize_database(self):
```

```
        """Create tables if they don't exist"""
        with self.get_connection() as conn:
            cursor = conn.cursor()
            cursor.executescript("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT UNIQUE NOT NULL,
                    email TEXT UNIQUE NOT NULL,
                    balance DECIMAL(10,2) DEFAULT 0.00,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                );

                CREATE TABLE IF NOT EXISTS transactions (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    from_user TEXT NOT NULL,
                    to_user TEXT NOT NULL,
                    amount DECIMAL(10,2) NOT NULL,
                    status TEXT DEFAULT 'PENDING',
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                );

                CREATE TABLE IF NOT EXISTS audit_log (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    action TEXT NOT NULL,
                    details TEXT,
                    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                );
            """)
            conn.commit()

    @contextmanager
    def get_connection(self):
        """Get database connection from pool (simulated)"""
        try:
            if not self.connection:
                self.connection = sqlite3.connect(self.db_path)
                self.connection.row_factory = sqlite3.Row


            self.connection.execute("SELECT 1")
            yield self.connection

        except sqlite3.Error as e:
            raise DatabaseError(f"Connection error: {e}")

    def transfer_money(self, from_user: str, to_user: str, amount: float) -> dict:
        """Transfer money between users with proper transaction handling"""

        with self.get_connection() as conn:
            with database_transaction(conn) as cursor:
                try:
```

```
            cursor.execute("SELECT balance FROM users WHERE username = ? FOR UPDATE",
(from_user,))
            sender = cursor.fetchone()

            if not sender:
                raise ConstraintViolationError(f"Sender {from_user} not found")

            if sender['balance'] < amount:
                raise ConstraintViolationError(
                    f"Insufficient balance: {sender['balance']} < {amount}"
                )


            cursor.execute("SELECT id FROM users WHERE username = ?", (to_user,))
            if not cursor.fetchone():
                raise ConstraintViolationError(f"Receiver {to_user} not found")


            cursor.execute("""
                INSERT INTO transactions (from_user, to_user, amount, status)
                VALUES (?, ?, ?, 'PROCESSING')
            """, (from_user, to_user, amount))
            transaction_id = cursor.lastrowid


            cursor.execute(
                "UPDATE users SET balance = balance - ? WHERE username = ?",
                (amount, from_user)
            )


            cursor.execute(
                "UPDATE users SET balance = balance + ? WHERE username = ?",
                (amount, to_user)
            )


            cursor.execute(
                "UPDATE transactions SET status = 'COMPLETED' WHERE id = ?",
                (transaction_id,)
            )


            cursor.execute("""
                INSERT INTO audit_log (action, details)
                VALUES (?, ?)
            """, ("TRANSFER", f"Transfer ${amount} from {from_user} to {to_user}"))

            return {
                "success": True,
```

```
                    "transaction_id": transaction_id,
                    "from": from_user,
                    "to": to_user,
                    "amount": amount
                }

            except ConstraintViolationError:
                raise

            except Exception as e:
                raise DatabaseError(f"Transfer failed: {e}")

    def batch_insert_users(self, users_data: List[Tuple]) -> dict:
        """Batch insert users with partial failure handling"""
        successful = []
        failed = []

        for i, user_data in enumerate(users_data):
            try:
                with self.get_connection() as conn:
                    with database_transaction(conn) as cursor:
                        cursor.execute(
                            "INSERT INTO users (username, email, balance) VALUES (?, ?, ?)",
                            user_data
                        )
                        successful.append(user_data[0])

            except sqlite3.IntegrityError as e:
                failed.append({"user": user_data[0], "error": str(e)})

            except Exception as e:
                failed.append({"user": user_data[0], "error": str(e)})

        return {
            "total": len(users_data),
            "successful": successful,
            "failed": failed,
            "success_rate": f"{(len(successful)/len(users_data)*100):.2f}%"
        }


def demonstrate_banking_system():
    repo = UserRepository(":memory:")


    with repo.get_connection() as conn:
        cursor = conn.cursor()
        cursor.execute("INSERT INTO users (username, email, balance) VALUES (?, ?, ?)",
                       ("alice", "alice@example.com", 1000))
        cursor.execute("INSERT INTO users (username, email, balance) VALUES (?, ?, ?)",
                       ("bob", "bob@example.com", 500))
```

```
            conn.commit()


    print("\n=== Test 1: Successful Transfer ===")
    try:
        result = repo.transfer_money("alice", "bob", 200)
        print(f"Transfer Result: {result}")
    except DatabaseError as e:
        print(f"Transfer failed: {e}")


    print("\n=== Test 2: Insufficient Funds ===")
    try:
        result = repo.transfer_money("bob", "alice", 1000)
        print(f"Transfer Result: {result}")
    except DatabaseError as e:
        print(f"Transfer failed: {e}")


    print("\n=== Test 3: Batch Insert ===")
    users = [
        ("charlie", "charlie@example.com", 300),
        ("alice", "alice_new@example.com", 400),
        ("david", "bob@example.com", 500),
        ("eve", "eve@example.com", 600)
    ]
    result = repo.batch_insert_users(users)
    print(f"Batch Insert Result: {result}")

if __name__ == "__main__":
    demonstrate_banking_system()
```

Interview Talking Points:

1. For the Order Processing System:

- **"I used custom exception hierarchy to differentiate between business logic errors and system errors"**
- **"The retry logic in payment processing handles transient failures"**
- **"Transaction rollback ensures data consistency across inventory, payment, and shipping"**
- **"Logging at each step helps with debugging in production"**

2. For the File Processing System:

- **"The retry decorator with exponential backoff handles API failures gracefully"**

- **"Failed files are moved to an error directory for manual inspection"**
- **"The system generates comprehensive reports showing success/failure rates"**

3. For the Database Transaction Manager:

- **"Context managers ensure connections are always properly closed"**
- **"Savepoints allow for nested transaction control"**
- **"Batch operations continue processing even if individual records fail"**
- **"Audit logging maintains an immutable record of all operations"**

Key Interview Questions You Can Answer:

Q: How do you handle partial failures?

- Show the batch processing example with individual error tracking

Q: What's your approach to retry logic?

- Point to the decorator pattern with exponential backoff

Q: How do you maintain data consistency during errors?

- Demonstrate the transaction rollback and compensating transactions

Q: How do you handle resources in case of exceptions?

- Show the context managers and finally blocks

These examples demonstrate production-ready patterns that interviewers look for: proper error classification, graceful degradation, comprehensive logging, and clean resource management.